[Wiki] Performance: https://wiki.haskell.org/index.php?title=Category:Performance

seq
Strict evaluation: https://www.haskell.org/onlinereport/haskell2010/haskellch6.html#x13-1260006.2
Prelude: https://hackage-content.haskell.org/package/base/docs/Prelude.html#v:seq

Prelude

```
seq :: a -> b -> b
($!) :: (a -> b) -> a -> b
```

Two main uses for forcing evaluations:
1) **performance**: eliminate **thunks** for performance
2) **error handling**: handle **bottom** as an error

usage 1: evaluate for **performance**

```
replicate' :: Int -> a -> [a]
replicate' 0 _ = []
replicate' n v = let xs = rep (n-1) v in xs `seq` (v:xs)        --
evaluate to avoid thunks
```

usage 2: evaluate for **error**
Use the **evaluate** function to throw a **bottom** as an error in **IO**.
https://hackage-content.haskell.org/package/base/docs/Control-Exception.html#v:evaluate

Haskell has no way to handle bottoms in a **pure expression**.
e.g. catch with evaluate

```
x :: ()
x = error ""

test :: IO ()
test = do
  result <- try @SomeException (evaluate x)        -- similar to
seq
  case result of
    Left e -> ...             -- handle error
    Right v -> ...            -- handle result
```

***) GOTCHA: strictness of data vs newtype**
[FPComplete] all about strictness: https://academy.fpblock.com/haskell/tutorial/all-about-strictness/
Laziness between data/newtype: https://stackoverflow.com/questions/13566673/laziness-strictness-between-data-and-newtype

data vs newtype

newtypes are **strict by default** since newtypes constructors are **unlifted**
i.e. if newtype A x = A x, then A x ~ x so A undefined ~ undefined
newtype: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-7400004.2.3

e.g.

```
data A a = A a            -- non-strict by default
data A' a = A' !a         -- strict
newtype B b = B b         -- strict by default

A undefined `seq` ()      -- ()
A' undefined `seq` ()     -- undefined
B undefined `seq` ()      -- undefined
```

e.g. real examples

Maybe	data
Identity	newtype

```
import Data.Functor.Identity

Just undefined `seq` ()           -- ()
Identity undefined `seq` ()       -- undefined
```

***) usage: strictness in different depths**

Depths in argument

```
f :: (a, b) -> (b, a)
f ~(x, y) = (y, x)                    -- lazy at top level
f undefined                           -- (undefined, undefined)
lazy

f (x, y) = (y, x)                     -- strict at top level only
f undefined                           -- undefined       strict
f (undefined, 1)                      -- (1, undefined)  lazy

f (!x, !y) = (y, x)                   -- strict at both levels
f undefined                           -- undefined  strict
f (undefined, 1)                      -- undefined  strict
```

Depths in result

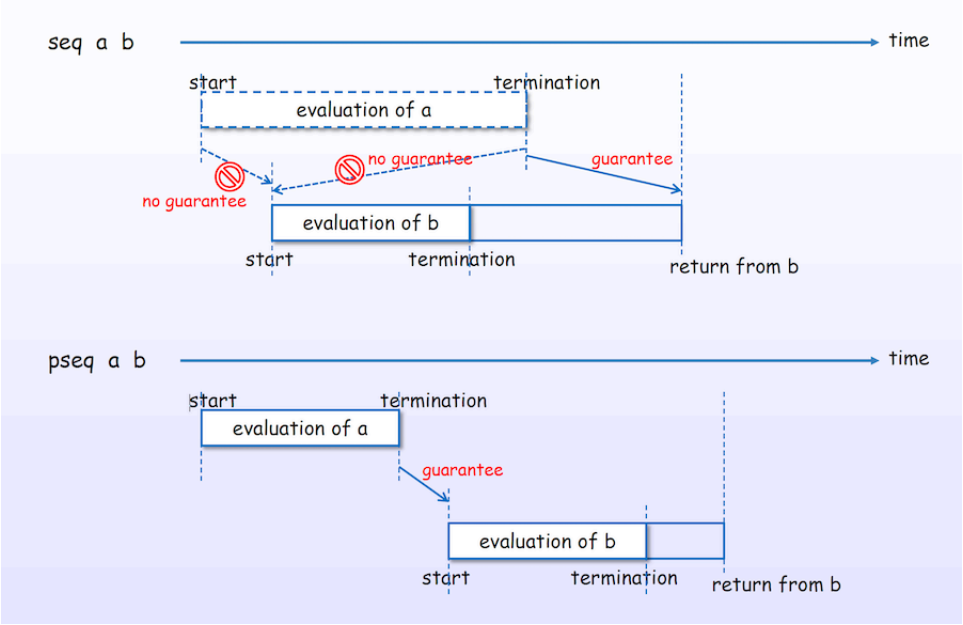
```
f :: a -> Maybe [a]
f !x = Just [a]               -- strict at top level of result
f undefined                   -- undefined  strict

f x = Just $ x `seq` [x]      -- lazy in Maybe, but strict in list
[a]
f undefined                   -- Just undefined  s
```

```
f x = Just $ [x `seq` x]      -- lazy
f x = Just $ [x]              -- same
f undefined                   -- Just [undefined]
```

***) Usage: seq vs Control.Parallel.pseq**
Control.Parallel: https://hackage-content.haskell.org/package/parallel/docs/Control-Parallel.html

pseq ensures that the first argument is always evaluated **first**
https://takenobu-hs.github.io/downloads/haskell_lazy_evaluation.pdf



e.g.

```
x = error "x"
y = error "y"
x `seq` y             -- may be error "x" or error "y"
x `pseq` y            -- always error "x"
```

***) Usage: seq vs deepseq**
deepseq: https://hackage.haskell.org/package/deepseq
seq is a **shallow** evaluation upto **weak normal head form (WNHF)** i.e. force evaluate only **one level**
deepseq is a **deep** evaluation upto **normal form** i.e. force evaluate **all** levels

```
import Control.Deepseq

-- seq
seq undefined ()                    -- ERROR
seq (Just undefined) ()             -- () i.e. no error
seq (Just (Just undefined)) ()      -- ()
```

```
-- deepseq
deepseq @Int undefined ()                    -- ERROR
deepseq (Just @Int undefined) ()             -- ERROR
deepseq (Just (Just @Int undefined)) ()      -- ERROR
```

***) Usage: Control.Exception evaluate (and evaluate vs seq/$!)**
Control.Exception evaluate: https://hackage-content.haskell.org/package/base/docs/Control-Exception.html#v:evaluate

evaluate

```
evaluate :: a -> IO a
return :: a -> IO a       -- for IO monad
```

Provides a way handle **bottoms** aka invalid expressions.
There is no way to handle bottoms as a **pure expression**.
e.g.

```
-- e.g. cannot pattern match
f :: Int -> Int
f undefined = 1               -- BAD. Matches everything
f _ = 2
f undefined                   -- 1
f 1                           -- 1

-- e.g. cannot use case
f = \case x of
  undefined -> 1              -- BAD Matches everything
  _ -> 2
f undefined                   -- 1
f 1                           -- 1
```

evaluate **throws** a pure expression as an error in **IO**.
This is accomplished by
a) **lifting** the pure expression to **IO**
b) **evaluating** the pure expression (to WHNF)
i.e. similar to **return :: IO a + seq**

```
evaluate  :: a -> IO a
return $! :: a -> IO a
```

e.g.

```
x = 1 `div` 0

test :: IO ()
test = do
  result <- try @SomeException (evaluate x)
  case result of
    Left e -> ...                                  -- Error case
```

```
    Right v -> ...                                      -- OK case
```

use **deepseq** to evaluate fully to normal form.

```
import Control.DeepSeq (force)

evaluate undefined                      -- throw
evaluate $ 1 + undefined                -- throw
evaluate $ Just undefined               -- no throw
evaluate $ force $ Just undefined       -- throw
```

evaluate vs return + seq or $!
return + seq does NOT guarantee that the evaluation happens i**nside IO**
Depends on execution e.g. optimizations
See https://hackage-content.haskell.org/package/base/docs/Control-Exception.html#v:evaluate

e.g. issues with return, return + seq, return $!

```
x = 1 `div` 0

test :: IO ()
test = do
  Right v <- try @SomeException (return x)              -- BAD. No
exception since x is NOT evaluated
  print v                                               --
EXCEPTION here

  Right v <- try @SomeException (return $ x `seq` x)    -- BAD. No
exception since seq is NOT evaluated yet
  print v                                               --
EXCEPTION here
  Right v <- try @SomeException (return x)              -- same
  print $ v `seq` v                                     --
EXCEPTION



  Left e <- try @SomeException (x `seq` return x)       -- BAD May
or may not work, depending on compiler optimization
                                                        -- i.e. it
may be either
                                                        --    enter
IO => error  OK  caught
                                                        --    error
-> no IO     BAD Not caught
  Left e <- try @SomeException (return $! x)            -- same
  print e
```

Why not seq or $!?
All have the same signature

```
evaluate x          :: IO a
return $ x `seq` x :: IO a
return $! x         :: IO a
x `seq` return x    :: IO a        -- same as $!
```

However, the internal is like

```
evaluate x           = IO $ \s -> x `seq` (# x, s #)       --
GOOD. seq on execution
return $ x seq x     = IO $ \s -> (# x `seq` x, s #)       -- BAD.
seq inside pair (#, #)
return $! x          = x `seq` IO (\s -> (# x, s #))       -- BAD.
seq outside of IO
```

Exact definition of evaluate

```
evaluate :: a -> IO a
evaluate a = IO $ \s -> seq# a s

-- where
seq# :: forall a s. a -> State# s -> (# State# s, a #)
seq# a s = let !a' = lazy a in (# s, a' #)                  --- seq
outside of (# , #)

-- laziness magic
lazy a = a
```

***) Usage: testing strictness with higher order functions i.e. functions that call functions**

Should use a strict function for testing, since lazy functions do not have a signal

e.g.

```
f :: a -> (a -> a) -> a   -- function to test
f x g = g x

l :: Int -> Int           -- lazy function
l _ = 1

f undefined id            -- undefined  GOOD since id is strict
f undefined l             -- 1          BAD
```

e.g. real world example with Writer

```
import qualified Control.Monad.Trans.Writer.CPS as C

-- OK: Sum x <> y is strict in both x and y
C.runWriter $ C.writer ((), undefined) >> C.writer ((), Sum 0)
    -- undefined
```

```
C.runWriter $ C.writer ((), Sum 0)     >> C.writer ((),
undefined)   -- undefined


-- BAD: list x <> y is strict only in x
C.runWriter $ C.writer ((), undefined) >> C.writer ((), "a")
    -- undefined
C.runWriter $ C.writer ((), "a")       >> C.writer ((),
undefined)   -- ((), ('a':undefined))
```

Lazy pattern matching
https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-580003.17
https://wiki.haskell.org/Lazy_pattern_match

Lazy pattern match
A Haskell **value** is evaluated **lazily**.
However patterns are still by default **strict**
i.e. evaluation continues only after the full pattern is determined.
e.g.

```
-- Normal lazy evaluation
g :: Int -> Int
g x = 0

g 1                             -- 0
g undefined                     -- 0 OK works by laziness


-- Lazy pattern match
f :: (a, b) -> Int
f (x, _) = 0
flazy :: (a, b) -> Int
flazy ~(x, _) = 0

f undefined                     -- EXCEPTION since pattern doesn't
match
flazy undefined                 -- 0  OK
```

Lazy pattern is equivalent to not using a pattern but an **accessor**.

```
f ~(a, b) = (a, b)        -- lazy pattern
f p = (fst p, snd p)      -- same
```

Hence

```
f :: (Int, ()) -> (Int, ())       -- lazy
f ~(a, _) = (a + 1, ())

g :: (Int, ()) -> (Int, ())       -- not lazy
```

```
g (a, _) = (a + 1, ())

f undefined                         -- (<thunk>, ())
g undefined                         -- undefined

f undefined `seq` ()                -- OK ()
fst (f undefined) `seq` ()          -- EXCEPTION since depends on
input
snd (f undefined) `seq` ()          -- OK () since doesn't depend on
input
g undefined `seq` ()                -- EXCEPTION
```

```
-- Basic
--
f ~(a, b) = undefined     -- e.g. pair
f ~(Just a) = undefined   -- e.g. Maybe
f ~a = undefined          -- SILLY. Lazy pattern match is
meaningless for single variables

let (a, b) = v            -- let does not require ~
f ~(a, b) = ...           -- function arg
case v of                 -- case
  ~(a, b) -> ...
(\~(a, b) -> ...          -- lambda


-- multiple
let (a, (b,c)) = undefined            -- no lazy
let ~(a, (b,c)) = undefined           -- lazy on outer pair
let (a, ~(b, c)) = (1, undefined)
let ~(a, ~(b, c)) = undefined


-- GOTCHA: lazy pattern MUST be the last case
-- since it always matches
f :: [Int] -> Int
f (a:as) = a              -- case: empty
f [] = 0                  -- case: non-empty

f :: [Int] -> Int
f ~(a:as) = a             -- matches all
f [] = 0                  -- BAD. Meaningless

f :: [Int] -> Int
f [] = 0                  -- OK. Matched first if non-empty
f ~(a:as) = a             -- matches all
```

GHC Pragma: UNPACK pragma
GHC: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/

pragmas.html#unpack-pragma

UNPACK
Field as **raw value** in memory i.e. unboxed type instead of a pointer.
i.e. field values are **duplicated**.
e.g.

```
data MyData = MyData { x::Int }
    x = MyData    --pointer-->    field x    --pointer-->    Int
value

# With UNPACK
data MyData = MyData { x:: {-# UNPACK #-} Int }
    x = MyData    --pointer-->    field x = Int value
```

```
data MyData = MyData {-# UNPACK #-} Float
data MyData = MyData {-# UNPACK #-} (!Int,!Int)
  -- any data constructor
data MyData = MyData {-# UNPACK #-} Float {-# UNPACK #-} String
 -- multiple
data MyData = MyData {-# UNPACK #-} Float String
 -- only Float is unpacked
data MyData = MyData {-# UNPACK #-} !Float
data MyData = MyData ( {-# UNPACK #-} !Int, !Int)
 -- ERROR. Cannot be nested.
data MyData = MyData { x:: {-# UNPACK #-} !Float }
 -- field
```

GHC Extension: Bang patterns (!)
GHC: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/strict.html
[FP] All about strictness: https://academy.fpblock.com/haskell/tutorial/all-about-strictness/

Bang patterns
Shorthand for applying seq
e.g.

```
-- no seq
f (...)


-- with seq
let x' = ... in seq x' f x'


-- with !
let !x' = ... in f x'
```

e.g. equivalent

```
add :: Int -> Int -> Int
```

```
add !x !y = x + y

add :: Int -> Int -> Int
add x y = x `seq` y `seq` x + y
```

```
{-# LANGUAGE BangPatterns #-}
---- case: in function argument
f !x = ...           -- in function argument
f (!x, !y) = ...     -- e.g. multiple

-- case: in let
let !x = ...          -- in let
let (!x, !y) = ...

-- case: in case
case v of
 !x -> ...


-- patterns
!x = ...                    -- value
Just !x = ...               -- data constructor
(!x : (!xs)) = ...          -- list constructor
[!x, !y, !z] = ...          -- list items
(!a, !b) = ...              -- tuple

-- OK, but meaningless
!(x:xs) = ...
!(a, b) = ...
```

***) [BangPattern] Usage: seq vs !**
! is syntactic sugar for seq (see above)

```
sumN :: Integer -> Integer
sumN = f 0
  where
    f v0 0 = v0
    f v0 n = f (v0 + n) (n - 1)
```

```
e.g.
sumN 10 = f 0 10
=> f (0 + 10) 9                                    v0+n: lazy
                                                   n-1:  non-lazy
due to pattern match
=> f ((0 + 10) + 9) 8
=> f (((0 + 10) + 9) + 8) 7
=> ...
=>  (...((0 + 10) + 9) + 8)...) + 2) + 1
=> 55
```

```
-- sumN with seq
sumNSeq :: Integer -> Integer
sumNSeq = f 0
  where
    f v0 0 = v0
    f v0 n = let v' = v0 + n in v' `seq` f v' (n - 1)
```

```
e.g.
sumNSeq 10 = f 0 10
=> f 10 9
=> f 19 8
=> f 27 7
=> ...
=> 55
```

```
-- sumN with !
sumNBang :: Integer -> Integer
sumNBang = f 0
  where
    f v0 0 = v0
    f v0 n = let !v' = v0 + n in f v' (n - 1)
```

e.g. benchmark

```
import Criterion.Main

test = do
  let x0 = 10 ^ 6
  defaultMain
    [ bench "sumN" (whnf sumN x0),
      bench "sumNSeq" (whnf sumNSeq x0),
      bench "sumNBang" (whnf sumNBang x0)
    ]
```

Output

```
benchmarking sumN
time                 1.050 s    (957.2 ms .. 1.174 s)
    -- slow
                     0.998 R²   (0.995 R² .. 1.000 R²)
mean                 978.1 ms   (873.5 ms .. 1.016 s)
std dev              70.33 ms   (6.079 ms .. 86.72 ms)
variance introduced by outliers: 20% (moderately inflated)

benchmarking sumNSeq
time                 695.9 ms   (694.5 ms .. 696.7 ms)
    -- fast
                     1.000 R²   (1.000 R² .. 1.000 R²)
mean                 691.8 ms   (689.4 ms .. 693.2 ms)
std dev              2.360 ms   (790.3 µs .. 3.103 ms)
variance introduced by outliers: 19% (moderately inflated)
```

```
benchmarking sumNBang
time                 651.8 ms   (649.7 ms .. 654.7 ms)
    -- fast
                     1.000 R²   (1.000 R² .. 1.000 R²)
mean                 650.5 ms   (650.1 ms .. 651.2 ms)
std dev              630.5 µs   (37.19 µs .. 831.7 µs)
    -- low std. WHY? Isn't ! and seq the same?
variance introduced by outliers: 19% (moderately inflated)
```

GHC Extension: Strict by default
GHC: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/strict.html#strict-by-default-data-types

Extensions:
- **StrictData:** type **fields** are strict by default
- **Strict**: patterns are strict by default

```
{-# LANGUAGE StrictData #-}
-- Strict by default
data MyType a = MyType a        -- strict i.e. same as MyType !a
data MyType a = MyType !a       -- same i.e. ! is redundant
data MyType a = MyType ~a       -- lazy i.e. same as MyType a



{-# LANGUAGE Strict #-}
-- case: pattern in function argument
f x = ...                  -- strict i.e. same as f !x
f !x = ...                 -- same i.e. ! is redundant
f ~x = ...                 -- lazy x

-- case: pattern in let expression
let x = ...                -- strict
let ~x = ...               -- lazy

-- case: pattern in case expression
case v of x -> ...         -- strict
case v of ~x -> ...        -- lazy
```

GHC Extension: Unboxed tuples + sums
GHC: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/primitives.html#unboxed-tuples

```
---- Tuple type
x = (# 1, 2 #) :: (# Int, Int #)
x = (# #) :: (# Int, Int #)                    -- empty
x = (# 1, 2, 3 #) :: (# Int, Int, Int #)       -- multiple
(# a1, a2, a3 #) = x                           -- a1=1 a2=2 a3=3
```

```
---- Sum type
-- i.e. algebraic sum
x0 :: (# Int | String #)
x1 :: (# Int | String #)
x0 = (# 1 | #)
x1 = (# | "hello" #)
(# v | #) = x0                -- v = 1
(# v | #) = x0                -- ERROR
(# | v #) = x1                -- ERROR
(# | v #) = x1                -- v = "hello"
```

==========

== strictness analysis ==

[GHC doc] - fstrictness flag: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/using-optimisation.html#ghc-flag-fstrictness
[GHC wiki] demand analysis: https://gitlab.haskell.org/ghc/ghc/-/wikis/commentary/compiler/demand

Demand analysis
Usage analysis
Evaluation strategy of a function **argument** can be optimized depending on the **demand**.
In particular: if an expression is used **at least once** aka **strict**, then it can be evaluated **first**
i.e. use **call-by-value**.
Conversely, if an expression is used **at most once**, then it can be **non-shared** to preserve memory
i.e. use **call-by-name**
Otherwise, use evaluate **lazily**
i.e. **call-by-need**

Analysis name	Demand	Cardinality	Evaluation strategy
strict Demand analysis	at least once	>= 1	call by value
maybe Usage analysis	at most once	<= 1	call by name i.e. no sharing
absent Usage analysis	never	0	remove (use placeholder)
lazy	unknown	>= 0	call by need

Cardinality

Label	Name	Cardinality

B	Bottom	-
A	Absent	0
L	Lazy Unknown	>= 0
S	Strict	>= 1
1	Once Strict + once	== 1
M	Maybe Lazy + once	<= 1

Cardinality syntax

```
   card :=       B | A | L | S | 1 | M

   demand :=     card subdemand
                 | card

   subdemand :=  card                                      polymorphic
type a
                 | P(demand, demand, ..)                   product
type e.g. (a, b)
                                                           typeclass
e.g. Num a
                 | C(card, subdemand, subdemand, ...)      function
call e.g. (a -> b)
```

Function demand syntax

```
  <d1><d2> ... <dn>[divergence]

  divergence :=  b                surely diverges
                 | x              surely diverges or throws an error
```

To test, use **-ddum-dmd-signatures**
e.g.
script

```
-- script.hs
module Main where

f1 :: Int -> Int
f1 x = x + x

f2 :: Int -> Int -> Int
f2 x y = x + y
```

check signature

```
$ stack  ghc -- -O2 script.hs   -ddump-dmd-signatures -fforce-
recomp
```

```
==================== Demand signatures ====================
:Main.main: <L>
Main.$trModule:
Main.f1: <S!P(L)>
Main.f2: <1!P(L)><1!P(L)>
```

e.g.

```
x :: Int
x                          1L
x + x                      SL  x demanded twice
if b then x else 0         ML  x demanded only in True case


-- case: function
f :: Int -> Int
f 1                        1C(1,L) f called once, result is lazy
let x = f 1                1C(1,L)
  in x + x
f 1 + f 1                  SC(1,L) f called twice with same argument
f 1 + f 2                  SC(S,L) f called with different arguments



-- case: function with multiple arguments
g :: Int -> Int -> Int
g 0                        1C(1,L) partial applied
g 0 1                      1C(1,1C(1,L)) called in both arguments
g 0 1 + g 2 3              SC(S,C(1,L)) called multiple times in
first argument, once per in second argument for each
g 0 1 + g 2 1              SC(S,C(1,L)) same
g 0 1 + g 0 3              SC(S,C(1,L)) same
let g' = g 0 in            1C(1,C(S,L)) g called once in first
argument, multiple times in second argument
 g' 1 + g' 3


-- case: Product type
x :: Identity a
Identity v                          1L
x                                   1L same even though Identity not
matched
case x of (Identity v) -> v         1L = 1P(1L)  Identity matched
once
case x of (Identity v) ->           SL = SP(1L)  Identity matched
twice
  case x of (Identity w) -> ..


-- case: Product type with Maybe
x :: Maybe a
```

```
x                              1L = 1P(L)
case x of                      1L  matched once
  (Just x) -> x
  Nothing  -> ...


-- case: Product type with multiple types
x :: (a, b)
(u, v) = u                     1P(1L,A)  Only first value used
(u, v) = v                     1P(A,1L)  Only second value used
(u, v) = [u,v]                 1P(1L,1L)


-- case: polymorphic type
x :: a
x                1L = 1P(L)     lazy since nothing known about
inner type
x <> x           1L multiple
```

Demand signature

```
addInt :: Int -> Int -> Int       <1L><1L> = <1P(L)><1P(L)>
addInt x y = x + y
add :: Num a => a -> a -> a       same
add x y = x + y

doubleInt :: Int -> Int           <SL> = <SP(L)>   x used exactly
twice
doubleInt x = x + x
double :: Num a => a -> a         same
double x = x + x


-- polymorphic type
id :: a -> a                      <1L>
fst :: (a, b) -> a                <1P(1L,A)>
seq :: a -> b -> b                <1A><1L>


-- product type
idMaybe :: Maybe a -> Maybe a
idMaybe x = x                       <1L>
idMaybe (Just v) = Just v           <1L> same
idMaybe Nothing = Nothing

maybe ::  b -> (a -> b) -> Maybe a -> b    <ML><MC(1,L)><1L>
maybe _ f (Just x) = f x                    ML               maybe
in b since result only for Nothing case
maybe y _ Nothing = y                           MC(1,L)     match
in f since function only for Just case
```

```
-- bottom
f :: a -> b                         <B>
f _ = undefined
f _ = error "Oops"                  same

f :: String -> a -> b               <S><B>
f s _ = error a
```

==========

== OLD ==

Weak head normal forms (WHNF)

e.g. test with seq

```
Just undefined `seq` ()
undefined `seq`()
```

expression	WHNF	
# Basic values		
# scalar		
undefined :: Int	undefined	
()	()	
1 :: Int	1	
# array		
undefined :: [Int]	undefined	
[undefined] ::	[undefined]	
[1]	[1]	
# pair		
(1, 2)	(1, 2)	
(1, undefined)	(1, undefined)	
(undefined, "")	(undefined, "")	
undefined :: (Int, String)	undefined	
# data type		
data A a b = A !a b		strict in a only
A 1 2	A 1 2	
A undefined 1	undefined	since strict in a
A 1 undefined	A 1 undefined	since non-strict in b
# case: new type		
# strict by default		
newtype A a = A a		
A undefined	undefined	
A 1	1	since newtype is unboxed i.e. A a ~ a for new types

function application

f x y = x + y	
f 1 1	2
f undefined 1	undefined

infix

1 + 1	2
1 + undefined	undefined
undefined + 1	undefined
Just 1 >> undefined	undefined
Just 1 >> Just undefined	Just undefined

function in a type

(1, 1 + undefined)	(1, 1 + undefined)
Just (1 + undefined)	Just (1 + undefined)

Bang pattern

f a !b = Just (a + b)	
f 1 2	Just (1 + 2)
f 1 undefined	undefined
f undefined 2	Just (1 + undefined)

Lazy pattern match

f (a, b) = Just (b + 1)		normal
fz ~(a, b) = Just (b + 1)		lazy pattern match NOTE: same as fz x = Just (fst x + snd y)
f (1, 2)	Just (2 + 1)	
f (1, undefined)	Just (undefined + 1)	
f undefined	undefined	
fz (1, 2)	Just (2 + 1)	
fz (1, undefined)	Just (undefined + 1)	
fz undefined	Just (undefined + 1)	

main idea: evaluate allows **handling** error **aka bottom** of **pure expressions**
In non-strict semantics, pure expressions NEVER **trigger** an error by default, since pure expressions are not evaluated.
Hence, pure expressions alone can NOT **handle** errors.
Thus, to handle a pure expression error, it needs to:
1) lifted to **IO** AND
2) trigger **evaluation**
i.e. similar to **return :: IO a + seq**

```
evaluate  :: a -> IO a
return $! :: a -> IO a
```

e.g.

```
x = 1 `div` 0

test :: IO ()
test = do
  result <- try @SomeException (evaluate x)
  case result of
```

```
    Left e -> ...                                       -- Error case
    Right v -> ...                                      -- OK case
```

Like seq, evaluate only evaluates to **weak head normal form (WHNF)**.
Use **force** of Control.DeepSeq for full evaluation to **normal form (NF)**.

```
import Control.DeepSeq (force)

evaluate x                -- WHNF
evaluate $ force x        -- NF
```

evaluate vs return + seq or $!
In particular, **seq** does NOT work since the value needs to be evaluated **inside** the IO monad.
1) **return :: IO a**: raise to IO only, no evaluation
2) **seq**: evaluation only

return + seq does NOT guarantee that the evaluate happens inside IO
Depends on execution e.g. optimizations
See https://hackage-content.haskell.org/package/base/docs/Control-Exception.html#v:evaluate

Indeed, **evaluate** uses **seq**, but **inside**

```
evaluate :: a -> IO a
evaluate a = IO $ \s -> seq# a s                        -- seq
inside IO

-- where
seq# :: forall a s. a -> State# s -> (# State# s, a #)
seq# a s = let !a' = lazy a in (# s, a' #)

lazy a = a
```

e.g. issues with return, return + seq, return $!

```
x = 1 `div` 0

test :: IO ()
test = do
  Right v <- try @SomeException (return x)              -- BAD. No
exception since x is NOT evaluated
  print v                                               --
EXCEPTION here

  Right v <- try @SomeException (return $ x `seq` x)    -- BAD. No
exception since seq is NOT evaluated yet
  print v                                               --
EXCEPTION here
  Right v <- try @SomeException (return x)              -- same
  print $ v `seq` v                                     --
```

```
EXCEPTION


  Left e <- try @SomeException (x `seq` return x)       -- BAD May
or may not work, depending on compiler optimization
                                                       -- i.e. it
may be either
                                                       --    enter
IO => error  OK  caught
                                                       --    error
-> no IO     BAD Not caught
  Left e <- try @SomeException (return $! x)           -- same
  print e
```